1. Problem

Our goal is to calculate the unique visitor count for each day and per IP address for the web app `www.example.com`.

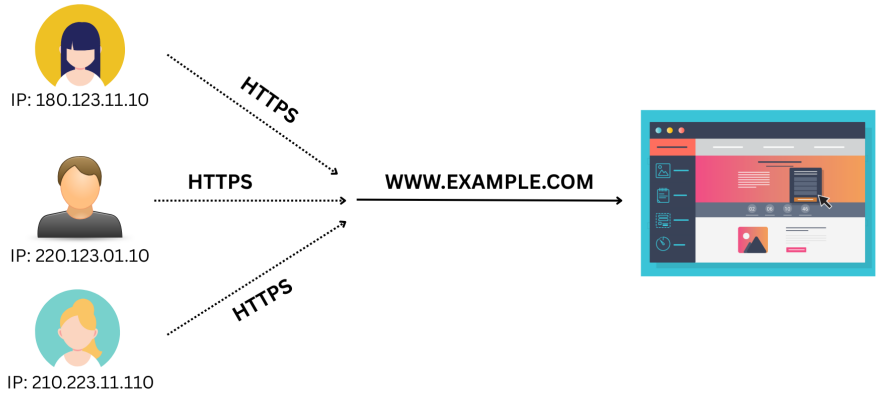

Figure 1: Multiple users with different IP addresses visiting `www.example.com`

Get all unique IP addresses from the session DB.

```
SELECT COUNT(ip) AS unique_users_count FROM session_db;
```

If we see this solution, we are actually scanning the full DB table to count the distinct number of users who visited our web app. For a million or billion rows `session_db` table, isn't this solution time-consuming?

What could be a better solution here?

It's Hyper Log Log. So let's see how the algorithm actually works for better understanding.

2. HyperLogLog Functions

We have two HyperLogLog functions: one for adding users to the bucket (`HLL.add(userIP)`), and another for getting the count (`HLL.count()`).

Now we will explore these two functions in detail.

3. HLL.add(userIP)

First, we will calculate a 64-bit binary hash of the user's given IP address. As we don't need cryptographic security here, we can use any lightweight, faster function that distributes the 0/1 binary value more effectively. e.g. `MurmurHash3`, `XXHash64`, etc.

```
const hashedUserIp = hash(userIp)
```

Now, based on the first n binary digits, we determine how many buckets to use to store the user's info. Choosing this value will tell us how much storage we need for buckets. For choosing the first 10 bits out of 64 bits, we will have $2^{10} = 1024$ buckets, each storing an integer and costing 1024×4 bytes, for a total of 4 KB of storage. So using only 4 KB storage, we can calculate approximately unique users who visited our site with just $\mathcal{O}(1024)$ time, which is nearly constant. But choosing 1024 buckets will result in a 3.25% error rate. By increasing the bucket size to 2^{16} and increasing the first $n = 16$ bits to reduce error rates, we will have 65,536 buckets, which will also increase the memory size to $65{,}536 \times 4$ bytes = 256 KB and the order of $\mathcal{O}(65{,}536)$, which is still faster than scanning the full million-row table.

3.1 Example 1 — IP `180.123.11.10`

Let's say we have:

$$\texttt{hashedUserIp} \leftarrow \texttt{Hash(180.123.11.10)}$$

which gives the following binary:

$$\underbrace{1010110010101101}_{\text{first 16 bits}}\underbrace{00100010100101101110001001011100010101100110101 10}_{\text{remaining 54 bits}}$$

Let's convert the first $n = 16$ digits `1010110010101101` into decimal to select the bucket number:

$$(1010110010101101)_2 = (44205)_{10}$$

So, for the user with IP `180.123.11.10`, it will be placed in `bucket[44205]`, where we can have at most 65,536 buckets.

In the next step, we will calculate the leading zeros of the remaining 54 bits.

`0010001010010110111000100101110001010110011010110`

Here, the remaining 54 values, there are 2 leading zeros, so we can put:

$$\texttt{bucket[44205]} = \max(\mathit{initial\_value},\ 2) = 2, \quad \text{where } \mathit{initial\_value} = 0$$

3.2 Example 2 — IP `220.123.01.10`

For the next user, let's assume:

$$\texttt{hashedUserIp} \leftarrow \texttt{Hash(220.123.01.10)}$$

which gives the following binary:

$$\underbrace{0000000000001010}_{\text{first 16 bits}}\underbrace{01101110100101101110001001011100010101110110101 1}_{\text{remaining 54 bits}}$$

Let's convert the first $n = 16$ digits `0000000000001010` into decimal to select the bucket number:

$$(0000000000001010)_2 = (10)_{10}$$

So, for the user with IP `220.123.01.10`, it will be placed in `bucket[10]`.

In the next step, we will calculate the leading zeros of the remaining 54 bits.

`01101110100101101110001001011100010101110110101 1`

Here, the remaining 54 values, there is 1 leading zero, so we can put:

$$\texttt{bucket[10]} = \max(\textit{initial\_value},\ 1) = 1, \quad \text{where } \textit{initial\_value} = 0$$

And we will have values from `bucket[0]` to `bucket[65535]` for all the visited users. We will keep adding users the same way by calling `HLL.add(userIP)`; it will store the results.

4. HLL.count()

Now we will jump to the next step, the next function `HLL.count()`.

Here, the first step is to calculate the combined strength (probability) of the pattern, which determines `bucket[10]` $= 2$, meaning $\frac{1}{2^2} = 25\%$; for another example, let's say `bucket[50]` $= 4$, the probability is $\frac{1}{2^4} = 0.1\%$. So `bucket[50]` has a rarer pattern than `bucket[10]`. So, in these steps, we are actually calculating the probability of a pattern, which shows how rare it is in a 54-bit sequence for leading zeros.

So for our `bucket[0]`, `bucket[1]`, ..., `bucket[10]`, ..., `bucket[50]`, ..., `bucket[44205]`, ..., `bucket[65535]`, we will calculate total strength:

$$S = \frac{1}{2^{M[0]}} + \frac{1}{2^{M[1]}} + \cdots + \frac{1}{2^{M[n-2]}} + \frac{1}{2^{M[n-1]}}$$

which is actually:

$$S = \sum_{k=0}^{m-1} \frac{1}{2^{M[k]}} = \sum_{k=0}^{m-1} 2^{-M[k]}$$

After finding the strength, we will calculate the actual result using the estimate function:

$$\boxed{\hat{n} = \frac{\alpha(m) \times m^2}{S}}$$

Where $\alpha(m)$ is the constant value called corrected bias, which is 0.7213. Check the reference's Section 3.2 to know how the 0.7213 is being calculated.

m is the bucket size; for us, it's 65,536, and S is the strength of the pattern we found in our remaining 54 bits, with a leading 0 for all 65,536 buckets.

Now, just return the estimated value as the count.

Reference

https://inria.hal.science/hal-00406166/file/dmAH0110.pdf